



Universidad Nacional de La Plata

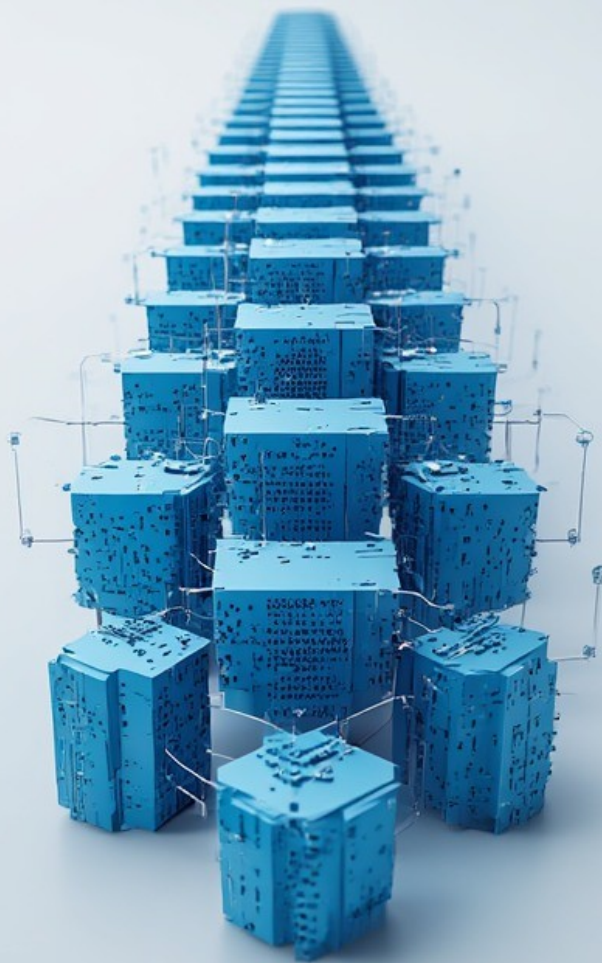


Facultad de Informática

Ejemplo de aplicación

Reducción en CUDA (No optimizado)





1

Reducción

2

Suma por reducción en GPU Nvidia

Estrategia. Solución CUDA.

An abstract graphic on the left side of the slide, featuring a dense, textured blue shape that resembles a cloud or a splash of paint, set against a white background. The shape is composed of many small, overlapping blue dots and lines, creating a sense of depth and movement. It is positioned on the left side of the slide, partially overlapping the white background.

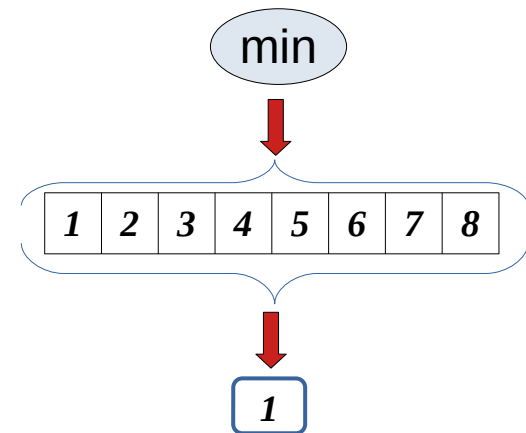
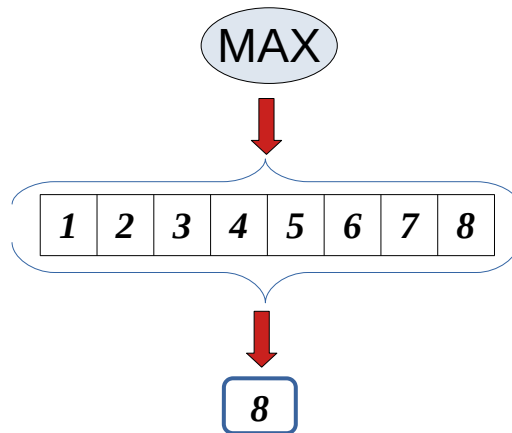
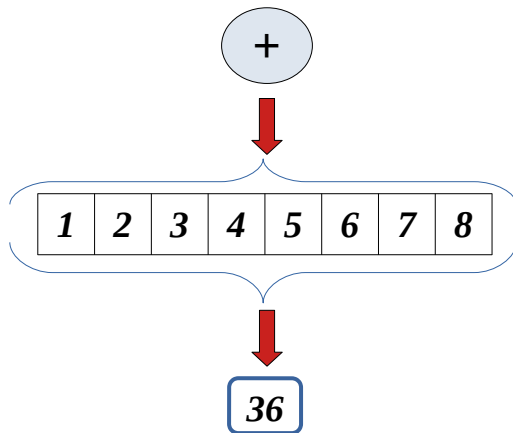
1

Reducción

Reducción

4

- **Reducción:** reducir los valores de un arreglo a un valor simple.
- Es posible hacerlo aplicando un operador asociativo:
 - +, *, MAX, min, etc.



A decorative graphic on the left side of the slide, consisting of a dense, blue, particle-like cloud or nebula shape, with a gradient from light blue to dark blue, and a soft shadow at the bottom.

2

Suma por reducción en GPU Nvidia

Estrategia

Solución CUDA

Suma por reducción

6

- La solución paralela tradicional requiere:
 - Distribuir el arreglo en partes entre los hilos.
 - ¿Conviene esta granularidad en GPU?
 - Cada hilo reduce a una suma parcial su porción almacenándola en una variable local.
 - Depende de la granularidad...(?)
 - Exclusión mutua para actualizar el valor global
 - ¡¡¡Imposible!!!



¿Cuál es la estrategia?



A decorative graphic on the left side of the slide, consisting of a dense, horizontal cloud of blue and white particles or dots, resembling a nebula or a data visualization, with a slight gradient from light blue on the left to darker blue on the right.

2

Suma por reducción en GPU Nvidia

Estrategia

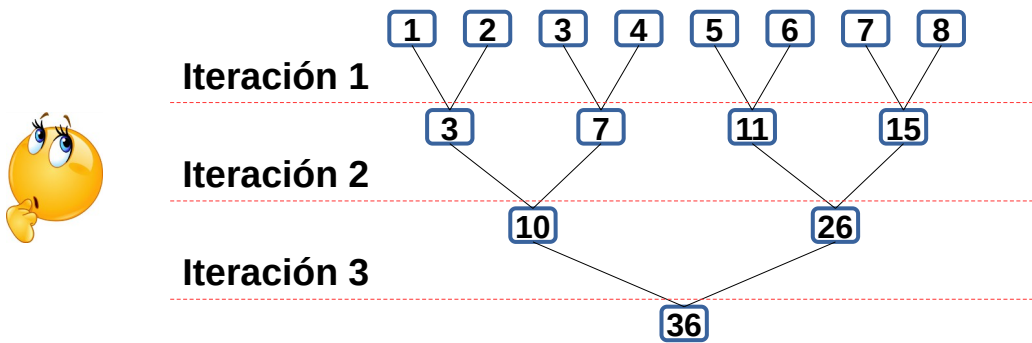
Solución CUDA

Suma por reducción

Estrategia

8

- Dado un vector de N elementos, la solución de una **suma por reducción** en GPU consiste en una implementación basada en árbol.
- Se requieren varias iteraciones, en cada iteración se realiza una nueva llamada al kernel y se aprovecha el hecho de que los valores de las variables en memoria global no cambian entre llamados al kernel.

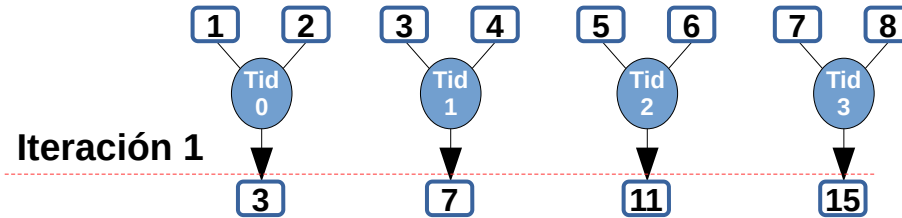


Suma por reducción

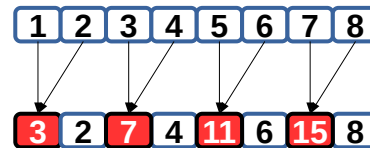
Estrategia

9

- Para la primera iteración se crean $N/2$ hilos.
- Cada hilo suma su posición y la siguiente.



- Para minimizar el espacio de almacenamiento, el resultado de cada suma parcial se almacena en la posición del primer operando.

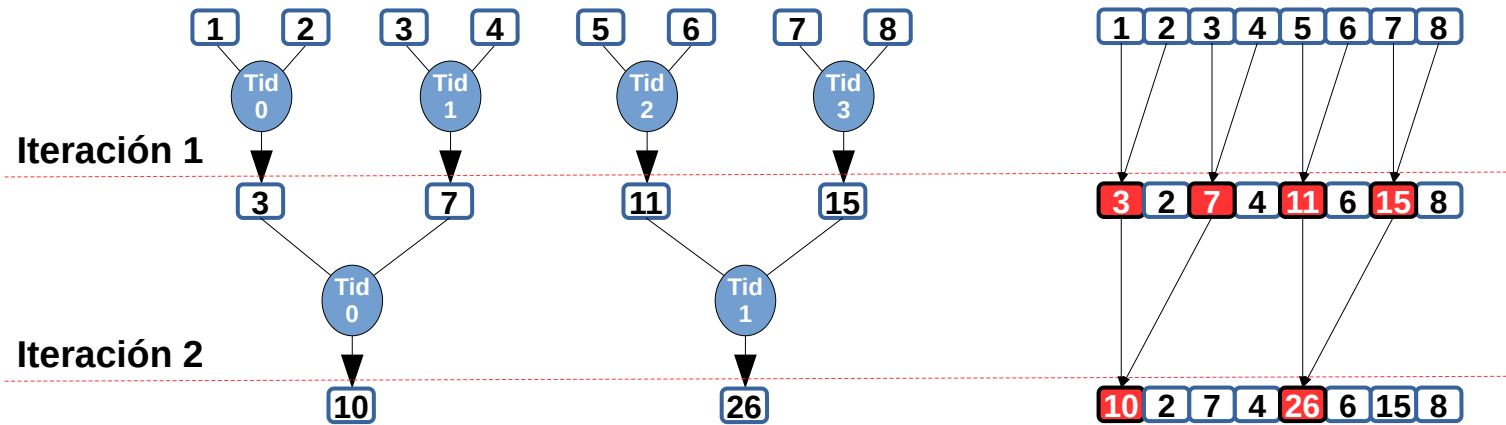


Suma por reducción

Estrategia

10

- Para la siguiente iteración se invoca nuevamente al kernel con la mitad de hilos de la iteración anterior y se sigue con la misma estrategia.

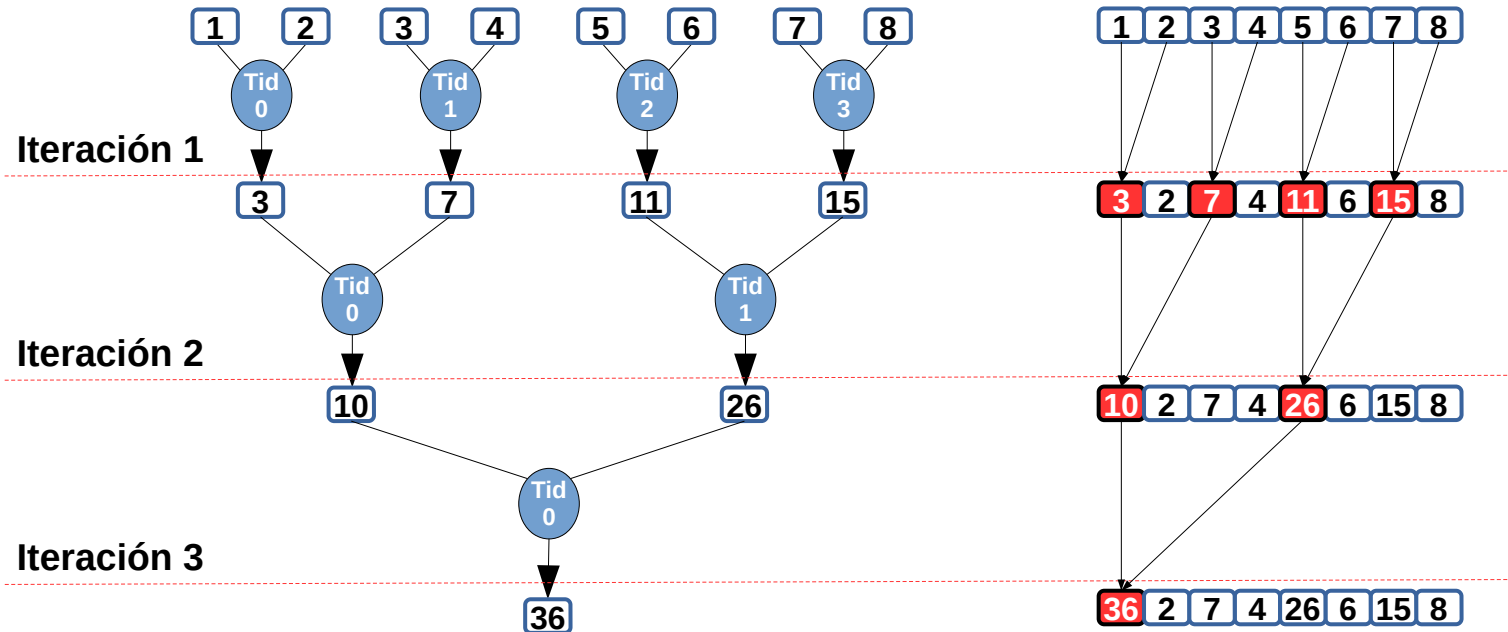


Suma por reducción

Estrategia

11

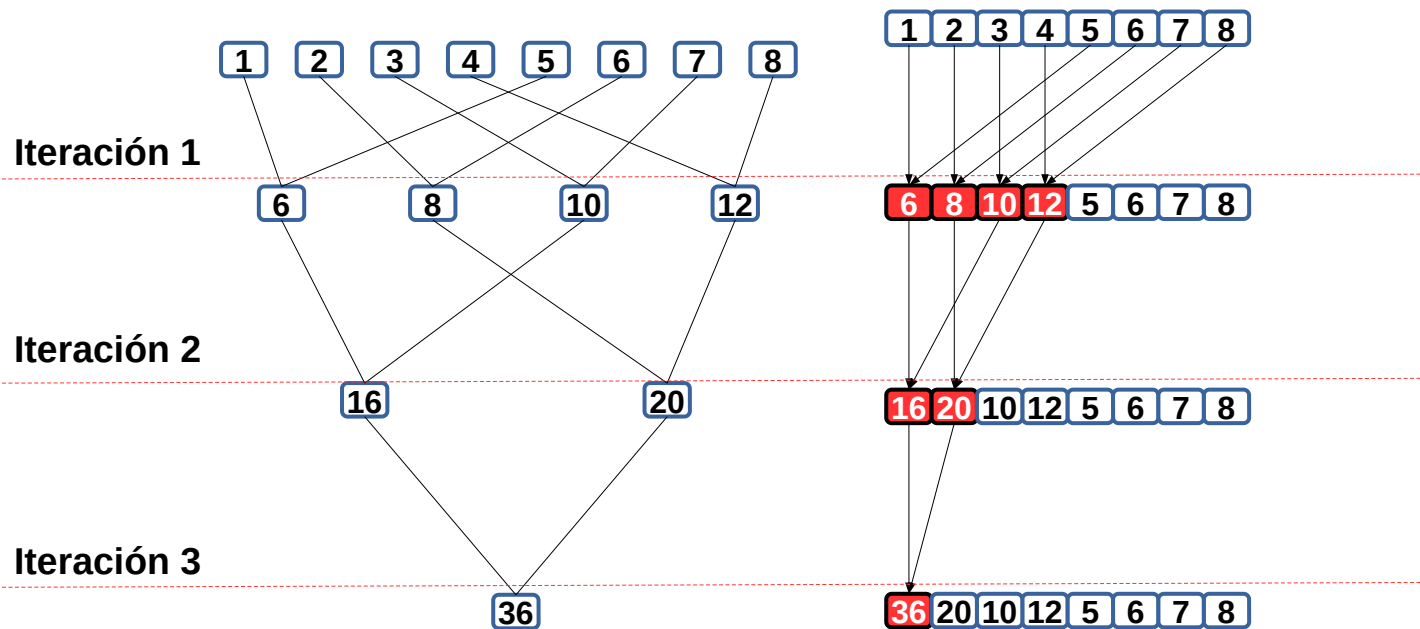
- En la última iteración se invoca al kernel con un sólo hilo, el resultado final queda en la primer posición del vector.



Suma por reducción

Alternativa para mejora de acceso a memoria

- Una alternativa, que mejora el acceso a memoria, es utilizar un patrón de acceso a posiciones contiguas.



A decorative graphic on the left side of the slide, consisting of a trail of blue particles or dots that form a shape resembling a comet or a stylized 'S' curve, pointing towards the bottom left.

2

Suma por reducción en GPU Nvidia

Estrategia

Solución CUDA

Suma por reducción

Solución CUDA

14

- Pseudocódigo:



???

```
__global__ kernelReduccion("parámetros"){  
    int idx = blockDim.x*blockIdx.x + threadIdx.x;  
    if (idx < "límite"){  
        vglobal[idx] += vglobal[idx + "distancia"];  
    }  
}
```

```
int main(int argc, char* argv[]){  
    int blockSize = 256;  
    dim3 dimBlock(blockSize);  
    ...  
    for ("nro iteraciones"){  
        dim3 dimGrid("en función de N y dimBlock")  
        kernelReduccion<<<dimGrid, dimBlock>>>("parámetros");  
        cudaDeviceSynchronize();  
    }  
    ...  
}
```

El tamaño del problema puede ser menor a la cantidad de hilos por bloque.
Un hilo podría leer una posición de memoria más allá de los límites del problema.



